1. What is a Robot?

Core Definition: An electromechanical system characterized by three essential traits:

- **Reprogrammable** Flexible software control
- **Multifunctional** Versatile multi-task capability
- **Sensible** Real-time environment awareness

Key Terminology:

- **Robot:** Physical mechanism performing human tasks autonomously or via teleoperation.
- **Robotics:** Science of mechanical design, kinematics, control & software.
- **Telerobotics:** Remote-controlled operation in extreme/hazardous environments.

2. Automation vs. Robots

Fixed Automation	Flexible Robots
Single dedicated task	**Diverse multiple tasks**
Maximum speed & efficiency	Reprogrammable motions
Rigid; high retooling cost	High mechanical flexibility
Ex: Bottling line, dishwasher	*Ex:* 6-DOF arms, AGVs, CNCs

3. Asimov's 3 Laws of Robotics

1. **Safety First:** May not injure a human or, through inaction, allow harm.
2. **Obedience:** Must obey human orders, **except** when conflicting with Law 1.
3. **Self-Defense:** Must protect own existence, unless conflicting with Laws 1 or 2.

4. Core Hardware Architecture

Sensors	**Data Input:** Internal state (encoders, resolvers) & external world (cameras, LIDAR, tactile/touch).
Controller	**The 'Brain':** Executes trajectories, computes kinematics & closes real-time feedback loops.
Actuators	**The 'Muscles':** Convert electrical/fluid energy into physical movement (servos, steppers, pneumatics).

5. Robot Types & Applications

Mechanical Architectures:

- **Manipulators:** Serial arms (Articulated, SCARA, Cartesian).
- **Mobile & Wheeled:** Differential drive, Omni, AGVs.
- **Legged:** Bipedal, quadrupeds, hexapods.
- **Autonomous Vehicles:** AUVs (Underwater), UAVs (Aerial drones).

Primary Drivers (3 D's / U's):

- **Dangerous** Nuclear cleanup, bomb disposal.
- **Dull / Repetitive** High-speed assembly, spot welding.
- **Dirty / Unwanted** Sewer inspection, toilet cleaning.

6. Robot Programming Methods

ON-LINE METHODS (At the Robot)

- **Teach Pendant:** Handheld button box. Jog arm to waypoints & save; controller calculates PTP paths.
- **Lead-Through:** Manually guide arm by hand. Records continuous stream (60–80 pts/sec; high memory).

OFF-LINE METHODS (Remote / Code)

- **Programming Languages:** Text code (AML, VAL, RobotStudio) without halting factory line.
- **Task-Level:** High-level goals ("pick item A") auto-compiled into trajectory commands.

7. Performance Metrics

- **Working Volume:** 3D envelope reachable by end-effector.
- **Speed & Accel:** Motion rate; trade-off with precision & payload.
- **Resolution:** Smallest commandable incremental step.
- **Accuracy:** Closeness between **commanded target** and **actual reached position** ($|x_{\text{target}} - x_{\text{actual}}|$).
- **Repeatability:** Ability to return to the **exact same point** over repeated cycles (σ).

HIGH-YIELD EXAM DISTINCTION

Accuracy vs. Repeatability: A robot can have **High Repeatability** (hits the identical spot 100 times in a row) but **Low Accuracy** (that spot is systematically 2 cm off target)!

1. Sensing & Transduction

- **Sensing:** Collecting information about the world.
- **Sensor:** Device mapping physical/chemical phenomena to quantitative signals.
- **Transduction Principle:** Converting one energy form into another.

Sensor Type	Transduction Mechanism
Thermistor	Temp → Resistance (R)
Photodiode	Light → Current (I)
Pyroelectric	Thermal rad. → Voltage (V)
Humidity	Moisture → Capacitance (C)
LVDT	Position → Inductance (L)
Microphone	Sound press. → Elect. signal

2. Sensor Classifications

State Focus:

- **Proprioceptive** (Internal): Measures internal parameters (joint angle, wheel position, battery, tachometers/encoders/accelerometers).
- **Exteroceptive** (External): Environment & objects (proximity, vision, range).

Energy Interaction:

- **Active:** Emits energy & reads reflection (Sonar, Radar, Modulated IR, LiDAR).
- **Passive:** Receives ambient energy only (Cameras, Pyroelectric, LDR).

Contact & Modality:

- **Contact / Non-contact:** Touch (bumpers) vs. remote (optical, ultrasonic).
- **Visual / Non-visual:** Vision cameras vs. non-optical sensors.

3. Contact & Resistive Sensors

A. FEELERS (TACTILE/CONTACT)

- **Whiskers:** Piano wire in metal hoop; deflection closes circuit; binary $(0/1)$ output.
- **Bumpers & Guards:** Frame on microswitches; collision triggers switch; binary $(0/1)$ output.

B. RESISTIVE (VARIABLE R)

- **Bend Sensors:** Flexible strips where R increases as bent ($10\ \mathrm{k}\Omega$ – $35\ \mathrm{k}\Omega$). *Uses:* Joint angles, wall-following, loads.
- **Potentiometers:** Linear/rotational variable resistors tracking sliding parts or rotating joint shafts.
- **Photocells (CdS / LDR):** Resistance shifts with light; highly non-linear, ideal for light tracking.

4. Capacitive vs. Inductive Sensors

Capacitive (All Materials)	Inductive (Metals Only)
• Measures change in **dielectric** • Detects **metals & non-metals** (liquids, plastics, powders) • Senses through glass/plastic • Sensitive to dust & humidity	• Emits AC magnetic field; senses **eddy currents** • Detects **metals ONLY** • Max on **ferrous** (iron/steel) • Immune to oil, dust & water
APPLICATIONS	**APPLICATIONS**
• **Level Detection:** Liquid/grains • **Proximity:** Assembly lines • **Touch Sensing:** Touchscreens	• **Position:** Metal part automation • **Speed:** Rotating gears/shafts • **Metal Detection:** Safety

5. Infrared (IR) Sensor Types

Type	Principle & Trade-offs	Applications
Intensity-Based *Reflective*	• Measures reflected light • **Pro** Cheap, simple • **Con** Ambient & color sensitive	• Line-following • Wall-tracking • Break-beams
Modulated IR *Proximity*	• Flashes at $32 - 45\ \mathrm{kHz}$ • Receiver demodulates signal • **Pro** Immune to ambient light	• Proximity sensing • Remote controls • Obstacle avoidance
IR Triangulation *PSD Ranger*	• Emitter + lens + **PSD** • Angle of return gives range • **Pro** Color & light immune	• Sharp GP2D02 $(10 - 80\ \mathrm{cm})$ • Distance map

6. Ultrasonic Sensors & Noise

- **Principle:** Ultrasound burst ($\approx 50\ \mathrm{kHz}$). Time-of-Flight (t):

$$D = v \cdot t \quad \Longrightarrow \quad d = \frac{v \cdot t}{2} \quad (v \approx 340\ \mathrm{m/s\ in\ air})$$

- **Limitations:**
  - **Bearing Uncertainty:** Wide beam spread (opening arc $\approx 30^{\circ}$).
  - **Speed Latency:** Sound is slow (e.g. $30\ \mathrm{m}$ round-trip takes $\approx 200\ \mathrm{ms}$).
- **Noise Issues:** Acoustic noise & sensor crosstalk.

7. Laser Range Finders (LiDAR)

- **Performance:** Long range ($2\ \mathrm{m} - 500\ \mathrm{m}$), high resolution ($10\ \mathrm{mm}$), fast scans ($13 - 40\ \mathrm{ms}$).
- **Robustness:** Narrow beam; immune to ambient light & specular reflections.

HIGH-YIELD EXAM COMPARISON

Inductive vs. Capacitive: Inductive senses **metals only**; Capacitive senses **all materials** (dielectric shift).
Ultrasonic vs. LiDAR: Sonar has wide **bearing uncertainty** ($\approx 30^{\circ}$) & latency; LiDAR provides millimeter angular precision.

1. Agency & Marr's 3-Level Theory

- **Agency:** Capacity of an entity (human or robot) to act intentionally, make choices, & exert control over actions and outcomes.
- **David Marr's Framework:** Analyzes information systems across 3 empirical levels:

Level	Question / Focus	Search & Rescue Robot Example
L1: Comp.	What goal / problem is solved?	Locate trapped survivors in low-visibility
L2: Algo.	What processes / IO represent it?	Steer toward heat & CO_2 gradients
L3: Impl.	How physically realized in hardware?	Angle vectors to heat centroids; wheel motors

KEY EXAM CONCEPT L1 & L2 are abstract & identical across biology/robots. Distinction appears **only at Level 3**.

2. Bio-Inspired Robotic Mappings

Biological Model	Robotic Mapping	Mechanism
Bat Echolocation	Obstacle Avoidance	Echo delay detection (ultrasonic)
Ant Foraging	Swarm Pathfinding	Ant Colony Optim. (ACO), digital pheromones
Bird Flocking	Coordinated Drone Swarm	Reynolds Boids: alignment, cohesion, separation
Frog Tongue	Fast Grasping Arm	High-speed camera + snatch reflex
Cuttlefish	Adaptive Camouflage	Pattern mapping onto E-skin

3. Classifications of Behavior

BEHAVIOR DEFINITION: Direct mathematical mapping of **sensory inputs** to **motor actions** to achieve a task.

1. Reflexive (Stimulus-Response) — Hardwired / Fastest

- **Reflex** Lasts **only during stimulus** (knee-jerk, pupil reflex).
- **Taxis** **Directional steering** relative to stimulus:
  - *Tropotaxis:* Turtles to moonlight | - *Phototaxis:* Worms away from light | + *Rheotaxis:* Salmon upstream
- **FAP** Response **persists longer** than stimulus (spider web, nut caching, duckling imprinting).

2. Reactive Behaviors — Muscle Memory

Learned skills consolidated to execute automatically without conscious thought (e.g. riding a bicycle).

3. Conscious Behaviors — Deliberative

Deliberate, goal-directed planning coordinating multiple sub-behaviors (e.g. puzzle solving, kit assembly).

4. Behavioral Schema Architecture

Behavioral Schema: feed

eyes ⇒ **Perceptual Schema** "small, moving" (feature filter) ⇒ location, intensity ⇒ **Motor Schema** "snap" (action calc) ⇒ vector

- **Frog Case Study:** Raw visual stimulus (eyes) is filtered into a clean percept (small, moving), passing coordinate (x, y) and intensity to the motor schema to generate a tongue **snap vector**.

5. Schema Theory & OOP Robotics

- **Schema:** Foundational OOP template used for the **Reactive Layer** of autonomous robots.
- **Components of a Behavior Schema:**
  - **Perceptual Schema** Sensory feature extraction, threshold filtering & stimulus delays.
  - **Motor Schema** Computational procedure to produce physical action/movement vector.
- **Schema Instantiation (SI):** Creating a concrete runtime instance from a generic parameterized class template (e.g. applying specific handlebar height & seat position to a bike-riding schema).

6. Vector Action & S-R Notation

Mathematical Formulations:

$$\{B : S \to R\} \quad \text{or} \quad B[S] = R$$

- S: Perceptual function converting raw sensor feeds to **percept**.
- R: Motor function converting percept into physical **action**.
- B: Overarching behavioral system.

Vector Field Combination:

- Directional stimuli (e.g. left & right eyes) combine via **Vector Summation** (Σ):

$$V_{\text{resultant}} = v_{\text{left}} + v_{\text{right}}$$

EXAM HIGHLIGHTS: LECTURE 03

Reflex vs. FAP Duration: Reflex stops as soon as stimulus stops; FAP continues for longer duration.
Taxis Examples: Positive Rheotaxis (salmon upstream), Tropotaxis (sea turtles to moonlight), Negative Phototaxis (earthworms).
Marr's Level 3: L1 & L2 are identical in biology & robotics; differences emerge **only at Level 3**.

1. LEGO Mindstorms EV3 Architecture

- **Processing Unit:** ARM9 @ 300 MHz, 16 MB Flash, 64 MB RAM.
- **Operating System:** Embedded Linux-based OS.
- **Sensor Ports (1-4):** 4 Input ports supporting Analog & high-speed Digital UART up to 460.8 kbit/s.
- **Motor Ports (A-D):** 4 Output ports with built-in optical rotation encoders for position/speed feedback.
- **I/O & Expansion:**
  - High-speed USB (480 Mbit/s) + Host daisy-chaining (up to 3 levels), WiFi dongle, & USB storage.
  - Micro SD-card slot (supports up to 32 GB).
- **Display & UI:** 178×128 monochrome LCD matrix, 6 backlit navigation/debugging buttons.
- **Wireless Comms:** Bluetooth v2.1 + EDR, Apple/Android smart device protocol.

2. Platform Generations & Modules

Platform	CPU / OS	Key Ports & Features
RCX (1998)	Hitachi H8	3 In / 3 Out; Optical IR tower comms
NXT (2006)	ARM7 + AVR	4 In / 3 Out; Bluetooth, 100×64 LCD
EV3 (2013)	ARM9 Linux	4 In / 4 Out; Daisy-chain, USB Host, SD

Sensor & Actuator Modules:

- **Motors:** Large (high torque) & Medium (high RPM) with integrated tacho-encoders.
- **Sensors:** Touch (binary limit), Color (ambient/reflected/RGB), IR Seeker/Beacon, Ultrasonic.

3. Step 1: Avoid Collision (Reflex)

PURE REFLEXIVE BEHAVIOR

- **Objective:** Move continuously; instantly back up and pivot when a physical impact occurs.

START →

Is Collision? (Touch == 1)

YES ↓	NO ↓
Reverse & Turn Motors B+C: -30, then steer	**Move Forward** Motors B+C: Power = 75

- **Collision == True:** Reverse (-30, 2 rot) → steer turn ($-45/30$, 2 rot).
- Direct hardware-to-motor S-R loop without state memory.

4. Step 2: Collision + Obstacle Avoidance

HIERARCHICAL PRIORITY ARBITRATION

- Combines contact (Touch) and non-contact (Proximity/Force) into a 2-tier reactive hierarchy:

1. Collision? (Touch)	2. Distance < 40cm?	
YES ↓	YES ↓	NO ↓
Reverse & Turn P1: Emergency Reflex	**Turn Away** P2: Proximity	**Forward** P3: Cruise

- **P1 (Touch):** Emergency physical recovery (overrides all).
- **P2 (Proximity):** Smooth steering turn away before contact.
- **P3 (Default):** Drive straight ahead.

5. Step 3a: 1-Sensor Line Following

BANG-BANG / EDGE TRACKING

- Tracks the **boundary edge** between light and dark surfaces using 1 photo/color sensor.

Reflected Light < Threshold?

YES (On Dark) ↓	NO (On Light) ↓
Turn Left Motor C ON / B OFF	**Turn Right** Motor B ON / C OFF

- $\mathrm{Ref} < \mathrm{Th}$ (On black) → Steer outside ($\mathrm{Turn\ Left}$).
- $\mathrm{Ref} \geq \mathrm{Th}$ (On white) → Steer inside ($\mathrm{Turn\ Right}$).
- **Control Characteristic:** 2-state discrete bang-bang controller; causes high-frequency zigzag oscillation along line edge.

6. Reactive Control Summary

Stage	Sensor Used	Behavior Pattern
Step 1	Touch Switch	Emergency reflex backoff
Step 2	Touch + IR/Dist	Layered priority avoidance
Step 3a	Color/Light	1-Sensor edge tracking

EXAM HIGHLIGHTS: LECTURE 04

EV3 vs NXT/RCX: EV3 introduced Linux OS, ARM9 @ 300MHz, daisy-chaining, USB host, & 4th motor port.
1-Sensor Line Following: Always tracks an **edge** (boundary transition), never the centerline itself.
Priority Arbitration: Safety/impact reflexes always take precedence over non-contact navigation.

1. Control Paradigm Taxonomy

- **Core Philosophy:** How sensing, decision-making, and execution interact.

Paradigm	Information Flow & Focus
Hierarchical / Deliberative	$\text{Sense} \to \text{Plan} \to \text{Act}$ Global world model, top-down plan
Reactive / Behavior-Based	$\text{Sense} \to \text{Act}$ Fast direct S-R loops, no world model
Hybrid	$\text{Plan} \to [\text{Sense} \to \text{Act}]$ Deliberative mission + reactive reflex

Hierarchical (Sense-Plan-Act):

Sense (Observe) → **Plan** (World Model) → **Act** (Execute)

- **'Eyes Closed' Execution:** Robot senses once, computes entire plan, then executes directives without re-evaluating until loop restarts.

2. Monolithic Sensing & Architecture

- **Monolithic Sensing:** All sensor observations fuse into **one global data structure** (World Model) accessed by planner:
  - **A Priori** Pre-loaded maps/building layout.
  - **Sensing Info** Current state & localization ("in NW hallway").
  - **Cognitive** Mission goals (deliver item to Room 118).

Autonomous Vehicle Pipeline:

Stage	Sub-Modules & Tasks
Sensors	Cameras, Radar, LiDAR, GPS / IMU
Perception	Obstacle Detection, Free Space, Localisation
Planning	Route → Behavioral → Trajectory
Control	PID, MPC → Drive-By-Wire (DBW) Actuators

3. Shakey & STRIPS Planning (6-Step Algorithm)

- **Shakey (1967-69, SRI / DARPA):** First AI mobile robot using **Means-Ends Analysis** (reduces state difference Δ).

The 6 Steps in Executing STRIPS — Recursive Stack

1. Compute difference ($\Delta = \text{Goal} - \text{Initial State}$) via evaluation function. If $\Delta = \varnothing$, terminate.
2. If $\Delta \neq \varnothing$, pick first operator from Difference Table whose **add-list** negates the difference.
3. Examine preconditions to find a set of variable bindings that evaluate all to **TRUE**.
4. **If FALSE precondition exists:** Make first FALSE precondition the **new subgoal**, push original goal to stack, & recurse (steps 2-3).
5. **When all preconditions match:** Push operator onto plan stack and update a copy of world model.
6. **Pop & Resume:** Return to parent operator with failed precondition to apply it or recurse on next failed condition.

4. STRIPS Predicates & Operator Anatomy

- **Predicates:** UPPERCASE statements evaluating to $\frac{\text{TRUE}}{\text{FALSE}}$ (e.g. $\text{INROOM}(\text{IT}, R_1)$, $\text{CONNECTS}(D_1, R_1, R_2)$).

Operator	Preconditions	Add-List	Delete-List
GOTODOOR (IT, dx)	$\text{INROOM}(\text{IT}, r_k)$ $\text{CONNECT}(d_x, r_k, r_m)$	$\text{NEXTTO}(\text{IT}, d_x)$	—
GOTHRU (IT, dx)	$\text{CONNECT}(d_x, r_k, r_m)$ $\text{NEXTTO}(\text{IT}, d_x)$ $\text{STATUS}(d_x, \text{OPEN})$	$\text{INROOM}(\text{IT}, r_m)$	$\text{INROOM}(\text{IT}, r_k)$

- **Generated Plan:** Stack sequence: $\text{GOTODOOR}(\text{IT}, D_1) \to \text{GOTHRUDOOR}(\text{IT}, D_1)$.

5. Warehouse STRIPS Case Study

Warehouse Layout: Locations Charging (C), Storage (S), Packing (P), Delivery (D).

Storage S	—— Path(S, P) ——	**Packing P**
| Path(S, C)		| **Blocked(P, D)** ✖
Charging C	—— Path(C, D) ——	**Delivery D**

Operator	Preconditions	Add-List	Delete-List
Move(x, y)	$\text{Path}(x, y)\ \neg\ \text{Blocked}(x, y)$ $\text{At}(\text{Robot}, x)$	$\text{At}(\text{Robot}, y)$	$\text{At}(\text{Robot}, x)$
PickUp(p, x)	$\text{At}(p, x)\ \text{At}(\text{Robot}, x)$ HandEmpty	$\text{Holding}(p)$	$\text{At}(p, x)$ HandEmpty
PutDown(p, x)	$\text{Holding}(p)\ \text{At}(\text{Robot}, x)$	$\text{At}(p, x)$ HandEmpty	$\text{Holding}(p)$

- **Obstacle Contingency:** Since $\text{Blocked}(P, D)$ is TRUE, planner avoids path $S \to P \to D$ and selects detour:

  Detour Plan: $\text{Move}(C, S) \to \text{PickUp}(P_1, S) \to \text{Move}(S, C) \to \text{Move}(C, D) \to \text{PutDown}(P_1, D)$

HIGH-YIELD EXAM DISTINCTION (LECTURE 05)

Deliberative Pros vs Cons: Pro: Computes optimal, goal-directed solutions. Con: Fragile under uncertainty, high computational complexity, slow to react to dynamic changes. **STRIPS Add/Delete Rule:** World state updates **only** by deleting predicates in Delete-List and appending predicates in Add-List upon successful precondition binding.